ConsumoZero

Global Solution 2026 - Coding for Security - Solução CitizenTech para reduzir desperdício de água, energia e alimentos

Aluno: Guilherme Botelho de Araújo - RM 569722 | Repositório: github.com/zhxofan/consumozero

1. Problema e cidadão

Cidadão afetado: famílias de baixa renda em bairros periféricos e pequenos comércios (mercadinhos, padarias, hortas comunitárias) da Região Metropolitana de Campinas. Eles pagam contas de água e luz que pesam no orçamento, jogam fora comida que venceu ou estragou e **não têm como saber se consomem acima do normal**: a conta chega, mas não diz se aquele número é alto, nem o que fazer.

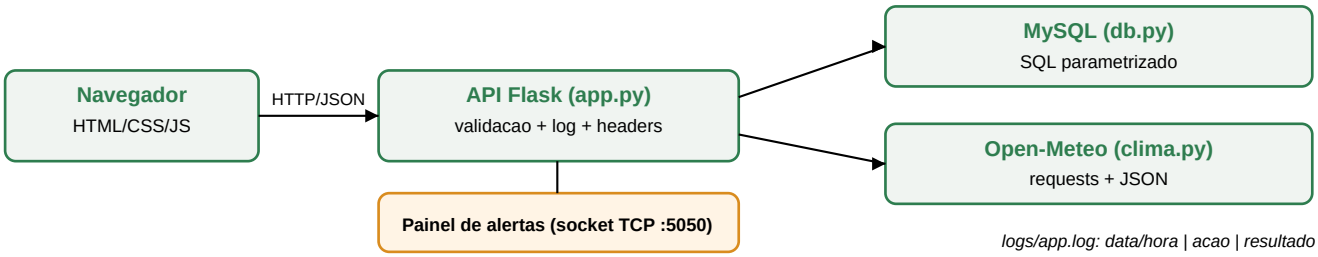
Evidências (dados públicos): (a) o PNUMA estima que cada brasileiro desperdiça em casa cerca de **94 kg de alimentos por ano** (Food Waste Index Report 2024); (b) o consumo médio residencial de energia no Brasil é de aproximadamente **161 kWh/mês** (EPE, PDE 2035), e o consumo residencial sobe em ondas de calor (EPE, Resenhas 2024); (c) a ONU recomenda cerca de **110 L de água por pessoa/dia**, enquanto o consumo esperado no país chega a ~175 L (Trata Brasil, 2025), com perdas na distribuição acima de 40%. **Observação de campo:** [descrever aqui uma conversa real - ex.: dono de mercadinho/vizinho, o que relatou sobre conta de luz ou descarte de alimentos].

Solução: o cidadão registra o consumo do mês por categoria usando só um apelido. O sistema compara com a referência pública ajustada ao número de moradores, mostra o percentual, dá uma dica prática, avisa um painel comunitário (ex.: associação de moradores) quando passa do limite e cruza com a previsão do tempo para alertar sobre dias de calor (mais consumo e mais comida estragando) ou de chuva (captação para limpeza e plantas).

2. ODS relacionados

ODS	Como a solução contribui
2 - Fome zero	Medir e reduzir o desperdício de alimentos libera comida e renda para a família ou para doação.
11 - Cidades e comunidades sustentáveis	Painel comunitário de alertas e uso eficiente de recursos urbanos (água e energia).
13 - Ação climática	Menos energia e menos alimento no lixo = menos emissões; alertas usam a previsão do tempo.
9 - Indústria, inovação e infraestrutura	Tecnologia simples e aberta, acessível a pequenos negócios.

3. Arquitetura e modelo de dados



Arquitetura em camadas: **app.py** (rotas e erros), **validacao.py**, **db.py** (CRUD sem ORM), **clima.py** (fonte externa), **alertas.py / alertas_server.py** (sockets TCP) e **logger.py**. Endpoints: GET /api/registros (200), GET /api/registros/<id> (200/404), POST (201/400), PUT (200/400/404), DELETE (204/404), GET /api/resumo e GET /api/clima (200/503). Erros de método e tamanho retornam 405 e 413.

Campo (tabela registros)	Tipo MySQL	Regra / motivo
id	INT AUTO_INCREMENT PK	Identificador
apelido	VARCHAR(40)	Regex 3-40 caracteres; substitui nome real (LGPD)
categoria / unidade	ENUM	agua-m3, energia-kWh, alimento-kg (unidade derivada no servidor)
quantidade	DECIMAL(10,2) + CHECK	Maior que 0 e até 100.000
mes_referencia	CHAR(7)	Formato AAAA-MM validado por regex
pessoas	TINYINT UNSIGNED + CHECK	1 a 50; ajusta a referência por pessoa
criado_em	DATETIME	Preenchido pelo banco

Uma única tabela foi suficiente para o protótipo e evita guardar qualquer vínculo entre a pessoa e o consumo além do apelido. Índices em categoria e mês aceleram filtros e o resumo.

4. Segurança e privacidade

Controles aplicados: credenciais no **.env** (fora do Git); **consultas parametrizadas** (%s) em todo o db.py; **validação por lista branca** de todos os campos (campos extras como CPF são recusados); mensagens de erro genéricas, com o detalhe só no log; usuário MySQL com **privilégio mínimo** (só SELECT/INSERT/UPDATE/DELETE na tabela); limite de 4 KB por requisição; **coleta mínima** (LGPD art. 6º, III - necessidade): apenas apelido e números de consumo.

Ameaça	Impacto	Mitigação implementada
SQL Injection (ex.: apelido = *x'; DROP TABLE registros;--*)	Perda ou vazamento de todos os registros	Queries parametrizadas no mysql-connector; regex no apelido; usuário do banco sem DROP/ALTER. Teste automatizado comprova a recusa.
XSS armazenado (script salvo no apelido e executado no navegador de outro usuário)	Roubo de sessão, páginas adulteradas	Validação por regex no servidor; front usa só textContent (nunca innerHTML); cabeçalho Content-Security-Policy default-src 'self'.
Vazamento de informação interna e de credenciais (stack trace, senha no código/Git)	Atacante descobre banco, host e senha	Credenciais no .env + .gitignore; FLASK_DEBUG=0; handlers globais devolvem mensagem genérica; teste verifica que o texto do erro interno não aparece na resposta.

Outros riscos tratados: abuso/DoS por corpo gigante (413), clickjacking (X-Frame-Options: DENY) e falha do serviço externo (timeout de 8 s e resposta 503, sem derrubar a API). O log nunca grava apelidos, só ação, id e status.

5. Validação e viabilidade

Testes automatizados: 17 testes com unittest (rodam sem MySQL usando um banco falso em memória) cobrindo o CRUD completo e os códigos 201, 200, 204, 400, 404, 405, 413, 500 e 503, tentativas de SQL Injection e XSS, campo extra (CPF), cabeçalhos de segurança e a leitura do JSON do Open-Meteo. Todos passando. Também foi testado o envio real de alerta pelo socket TCP entre cliente e servidor.

Validação com usuário: [registrar aqui o teste com 1-2 pessoas reais: quem usou, o que entendeu, o que sugeriu]. **Viabilidade:** usa apenas software livre e uma API pública gratuita; roda em um computador simples ou servidor barato e pode ser hospedado por uma associação de moradores ou ONG. Custo de dados: zero, pois não exige sensores.

6. Limitações

Os dados são digitados manualmente (dependem da leitura da conta e de pesar o lixo orgânico); a referência de energia é por residência e não se aplica bem a comércios; não há autenticação, então qualquer pessoa com acesso à rede pode editar registros (próximo passo: login com hash de senha e controle por apelido); falta HTTPS em produção; e a previsão do tempo usa coordenadas fixas definidas no .env.

7. Recursos além do conteúdo (bônus) e uso de IA

Testes com unittest.mock - permitem provar segurança e status HTTP sem depender do banco; **cabeçalhos de segurança HTTP (CSP, nosniff, X-Frame-Options)** - defesa em profundidade contra XSS e clickjacking; **usuário MySQL de privilégio mínimo e CHECK constraints** - mesmo que a aplicação falhe, o banco limita o estrago; **threads no servidor TCP** - o painel atende vários alertas ao mesmo tempo.

Declaração de IA generativa: foi usado o Claude (Anthropic) para apoiar a estruturação do código, a revisão de segurança, a escrita dos testes e a organização deste relatório. Todo o código foi revisado, executado e é explicado pelo autor.

Fontes: PNUMA, Food Waste Index Report 2024; Embrapa (27/03/2024); EPE, PDE 2035 - Caderno de Demanda de Eletricidade e Resenhas Mensais 2024; Instituto Trata Brasil, Estudo de Perdas de Água 2025 e Demanda Futura por Água em 2050; ONU (110 L/pessoa/dia); Open-Meteo API.